Solving the N-Queens Problem with SAT, CP, and MIP

Nguyen Phan Viet Hung

Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay Ward, Hanoi, 100000, Vietnam

ARTICLE INFO

Keywords:
N-Queens
SAT solving
at-most-one encodings
unit propagation
constraint programming
mixed-integer programming

ABSTRACT

When N-Queens is modelled with one Boolean variable per cell, it is necessary to state that each row, column, and diagonal contains at most one queen. This study compares five standard encodings of this constraint—pairwise, binary, commander, product, and sequential counter—and examines how the choice affects the cost of constructing and solving the formula. We derive expectations from their structure and evaluate them in 1,200 attempts at board sizes 8–128, using four SAT solvers and four native CP/MIP models. Loading time closely follows clause-count growth: doubling the board size multiplies pairwise loading by 8.0, consistent with its cubic clause count, and the other encodings by 3.8–4.7. Propagation work also differs substantially: with Glucose3 at $N = 64$, pairwise requires 8 propagations per decision and sequential 294. Glucose3–sequential is the only configuration to time out, at $N = 128$. Its long propagation chains offer a structural explanation for this difficulty. When search is inexpensive, as it is for CryptoMiniSat here, the encoding ranking follows preparation cost, with product leading. The best encoding varies by solver: commander for Glucose3, binary for CaDiCaL, and product for Kissat and CryptoMiniSat. Two further patterns suggest a role for the problem's structure: several configurations become *easier* as N grows, while CaDiCaL often uses about 6,000 conflicts across encodings and sizes. The abundance of N-Queens solutions and the solver's scheduled rephasing offer possible explanations, respectively; neither mechanism is directly measured. Native models are fastest overall under the worker wall-time metric, with CPLEX MIP leading.

1. Introduction

The N-Queens problem requires placing N queens on an $N \times N$ board such that no two queens share a row, column, or diagonal. A SAT solver cannot express this condition directly, since its input consists only of clauses. The requirement that each line contain at most one queen must therefore be compiled into an *at-most-one* (AMO) constraint, for which many encodings exist. Although this choice may appear to be an implementation detail, it determines the size of the formula, the number of steps required to derive consequences, and the variables on which the solver may branch.

This study examines that choice empirically. We consider five well-known AMO encodings—pairwise, binary, commander, product, and sequential counter—and pair each with four CDCL solvers: Glucose3, CaDiCaL, Kissat, and CryptoMiniSat. For comparison, we also solve the problem with models that require no explicit AMO encoding: IBM CP Optimizer, CPLEX MIP, Gurobi MIP, and OR-Tools CP-SAT.

To connect representation to performance, Section 4 examines formula size and propagation paths and derives expectations from them. Section 6 tests them by separating the costs of building the formula, searching it, and running the surrounding program. The main findings are:

1. **Building the formula.** Clause-count growth closely predicts loading-time growth. Pairwise, the simplest encoding, grows cubically, which makes it costly at large N.
2. **Searching it.** Once a queen is placed, all five encodings let unit propagation exclude the other cells on that line, but through paths of different lengths. These differences help account for Glucose3's propagation work and its timeout with sequential. The other solvers are less sensitive to the encoding, and some search patterns also suggest an influence from the structure of N-Queens itself.
3. **Running the program.** Starting a worker process introduces an additional cost. At the sizes tested, this overhead exceeds most SAT search times and contributes to the advantage of the native implementations.

The evidence is two complete batches of 600 attempts each, collected on 19 October 2026 under 2048- and 6144-MiB memory caps, with five repetitions at $N \in \{8, 16, 32, 64, 128\}$ and a 120-second deadline. The two caps give identical outcomes, so we report the 2048-MiB batch in the main text and document the comparison in Appendix B. Unrestricted N-Queens has known explicit constructions and differs from the NP-complete completion problem [6];

we use it as a controlled, well-understood workload in which representation effects can be observed clearly, rather than as a difficult problem in its own right.

2. Background

2.1. CNF formulas and unit propagation

A SAT solver takes a formula in conjunctive normal form (CNF): a conjunction of clauses, each a disjunction of literals x or $\neg x$. The solver's basic inference rule is *unit propagation* (UP): when all literals of a clause but one are false under the current partial assignment, the remaining literal is set true. Propagation repeats until no clause is unit or some clause becomes false, which is a *conflict*. How quickly a solver can rule out bad placements therefore depends on which literals UP can derive from an encoding, and on how many steps it needs to derive them.

2.2. At-most-one constraints and their encodings

For a set of variables $X = \{x_1, \dots, x_m\}$, the at-most-one and exactly-one constraints are

$$\mathrm{AMO}(X) \equiv \sum_{i=1}^{m} x_i \leq 1, \qquad \mathrm{EO}(X) \equiv \sum_{i=1}^{m} x_i = 1. \tag{1}$$

They are the simplest cardinality constraints, and their CNF encodings have been studied extensively [4, 5, 8, 15]. An encoding is a CNF formula $E(X, Y)$ that may introduce auxiliary variables Y; it is correct if an assignment of X satisfies $\mathrm{AMO}(X)$ exactly when it can be extended to a satisfying assignment of E. Its size is the number of clauses $|E|$ and of auxiliary variables $|Y|$.

Correctness alone does not determine how well a solver can reason with an encoding. For an isolated AMO constraint, the relevant property is propagation on its original inputs: whenever an input x_i becomes true, UP should derive $\neg x_j$ for every other input x_j, and hence detect a conflict if two inputs are true. This property maintains generalised arc consistency on the original AMO inputs. It does not imply propagation completeness under arbitrary auxiliary assignments or for the full N-Queens formula. Encodings with this property can still differ in the number of propagation steps between x_i and $\neg x_j$. We call the largest such number over all pairs $i \neq j$ within an isolated AMO constraint the *propagation depth* δ_E of the encoding.

The encodings studied here trade the quadratic clause count of the direct encoding for auxiliary variables. Table 1 lists the sizes of their standard constructions; Section 4 specifies the variants that we implemented, which differ in some local choices. Implementations of these encodings are available in libraries such as PySAT [7] and PBLib [12].

Table 1
Sizes of the standard AMO constructions on m inputs, as reported in the literature. The commander figures assume recursive groups of three; the product figures assume the recursive construction.

Encoding	Clauses $\lvert E\rvert$	Auxiliary variables $\lvert Y\rvert$	Source
Pairwise	$m(m-1)/2$	0	—
Binary	$m\lceil \log_2 m \rceil$	$\lceil \log_2 m \rceil$	Frisch and Giannaros [5]
Commander	$\approx 3.5\,m$	$\approx m/2$	Klieber and Kwon [8]
Product	$2m + 4\sqrt{m} + O(m^{1/4})$	$2\sqrt{m} + O(m^{1/4})$	Chen [4]
Sequential	$3m - 4$	$m - 1$	Sinz [15]

2.3. Conflict-driven clause learning

The four SAT solvers implement conflict-driven clause learning (CDCL), whose modern form descends from Chaff [10], which introduced watched-literal unit propagation and the VSIDS decision heuristic. A CDCL solver alternates *decisions*, which assign an unassigned variable at a new decision level, with unit propagation. When propagation produces a conflict, the solver derives a learnt clause C by resolution, adds it to the formula, and backjumps to an earlier level. Each assigned literal ℓ carries the decision level $\mathrm{dl}(\ell)$ at which it was assigned. Glucose [1, 2] ranks learnt clauses by their *literal block distance*

$$\mathrm{LBD}(C) = \Big|\{\mathrm{dl}(\ell) : \ell \in C\}\Big|, \tag{2}$$

retaining clauses with small LBD; it performs little simplification during search. CaDiCaL and Kissat [3] interleave search with *inprocessing*, which replaces the current formula F by a simpler equisatisfiable formula F', and periodically reset the saved phase vector $\varphi : V \to \{0,1\}$ used to choose decision values (*rephasing*); in some modes the new phases are taken from a short local-search walk that reduces the number of falsified clauses. CryptoMiniSat [16] adds further simplification and reasoning passes to a CDCL core. All four are accessed through PySAT [7]. Section 6 reports, where available, the numbers of decisions D, conflicts K, and propagations P, and uses the ratio P/D as a measure of propagation work per decision.

2.4. Properties of N-Queens

N-Queens has a long history as a benchmark for search and constraint algorithms [13]. Let $Q(N)$ denote the number of solutions of the N-Queens problem. Simkin [14] shows that

$$Q(N) = \left((1 + o(1))\, N e^{-\alpha}\right)^N, \qquad \alpha \approx 1.944, \tag{3}$$

so the number of solutions grows super-exponentially. Since every solution is a permutation of the columns and $N! = \left((1 + o(1))\, N/e\right)^N$, the fraction of permutations that are solutions satisfies

$$\frac{Q(N)}{N!} = \left((1 + o(1))\, e^{1-\alpha}\right)^N, \qquad e^{1-\alpha} \approx 0.389, \tag{4}$$

and therefore decreases exponentially. Neither quantity directly measures how difficult it is for a solver to reach a solution.

Writing a candidate placement as $q \in \{0, \dots, N-1\}^N$, with q_r the column of the queen in row r, the number of attacking pairs is

$$h(q) = \left|\{(r, r') : r < r',\ q_r = q_{r'} \ \vee\ |q_r - q_{r'}| = r' - r\}\right|, \tag{5}$$

and q is a solution if and only if $h(q) = 0$. Min-conflicts local search [9] repeatedly moves a queen to the column that minimises h, and local search with conflict minimisation solves boards with millions of queens [17]. These properties concern the unrestricted problem; N-Queens completion, in which some queens are preplaced, is NP-complete [6], and our conclusions do not extend to it.

3. Models of the problem

3.1. MIP model

Let $R = C = \{0, \dots, N-1\}$ be the row and column indices, and let $x_{r,c} \in \{0,1\}$ indicate a queen on cell (r, c). With diagonal sets $D_k^+ = \{(r,c) : r + c = k\}$ and $D_k^- = \{(r,c) : r - c = k\}$, a placement satisfies

$$\begin{aligned}
\sum_{c \in C} x_{r,c} &= 1 && \forall r \in R,\\
\sum_{r \in R} x_{r,c} &= 1 && \forall c \in C,\\
\sum_{(r,c) \in D_k^+} x_{r,c} &\le 1 && \forall k \in \{0, \dots, 2N-2\},\\
\sum_{(r,c) \in D_k^-} x_{r,c} &\le 1 && \forall k \in \{-(N-1), \dots, N-1\}.
\end{aligned}$$

CPLEX MIP and Gurobi MIP solve this model with a constant objective. Singleton diagonals are omitted, giving N^2 variables and $6N - 6$ constraints for $N \ge 2$. The row and column equalities alone describe the assignment polytope, whose vertices are exactly the permutation matrices (Birkhoff–von Neumann); only the diagonal inequalities cut against integrality. An LP-based solver therefore starts from a relaxation that already encodes "one queen per row and column" exactly.

3.2. CP model

Let $q_r \in \{0, \dots, N-1\}$ be the column of the queen in row r. Then

$$\text{AllDifferent}(q_0, \dots, q_{N-1}), \tag{6}$$

$$\text{AllDifferent}(q_0 + 0, \dots, q_{N-1} + N - 1), \tag{7}$$

$$\text{AllDifferent}(q_0 - 0, \dots, q_{N-1} - (N-1)). \tag{8}$$

Rows are enforced by the variable domains, and columns and diagonals by three global constraints. IBM CP Optimizer and OR-Tools CP-SAT use this model: N variables and three constraints, against tens or hundreds of thousands of clauses for SAT at $N = 128$. CP-SAT is a lazy clause generation solver [11], so internally it translates parts of this model into Boolean reasoning; CP Optimizer propagates the global constraints directly.

3.3. SAT model

The SAT model uses the MIP variables with identifier $\text{id}(x_{r,c}) = rN + c + 1$. With $\text{ALO}(X) = \bigvee_{x \in X} x$ and $\text{EO}(X) = \text{ALO}(X) \wedge \text{AMO}(X)$, the formula is

$$F_N = \bigwedge_{r \in R} \text{EO}(\{x_{r,c} : c \in C\}) \wedge \bigwedge_{c \in C} \text{EO}(\{x_{r,c} : r \in R\}) \wedge \bigwedge_{D \in \mathcal{D}} \text{AMO}(\{x_{r,c} : (r,c) \in D\}), \tag{9}$$

where $\mathcal{D}$ holds both diagonal directions with at least two cells. There are $6N - 6$ AMO constraints of length at most N; the encodings differ only in how each AMO is expressed as clauses. Auxiliary variables of different constraints are disjoint and numbered above N^2. The positive column clauses are logically redundant (Appendix D); both batches include them.

4. At-most-one encodings

Let $X = \{x_1, \dots, x_m\}$ be the inputs of one AMO constraint. Pairwise, binary, and sequential use PySAT `CardEnc`; commander and product are custom implementations whose local choices we state exactly, because the measurements describe these formulas and not the encoding families in the abstract.

4.1. The encodings

Pairwise. The pairwise encoding states directly that no two inputs are both true:

$$\text{AMO}_{\text{pw}}(X) = \bigwedge_{1 \le i < j \le m} (\neg x_i \vee \neg x_j). \tag{10}$$

It requires no auxiliary variables but $\binom{m}{2}$ clauses. Over the whole board, including the $2N$ positive row and column clauses,

$$C_{\text{pw}}(N) = 2N + 2N\binom{N}{2} + 2\binom{N}{2} + 4\sum_{m=2}^{N-1}\binom{m}{2} = \frac{5N^3 - 6N^2 + 7N}{3}, \tag{11}$$

which is *cubic* in N. (This encoding is called `binomial` in the implementation.)

Binary. Each input x_i is assigned a distinct ℓ-bit code $c(i) \in \{0,1\}^\ell$, where $\ell = \lceil \log_2 m \rceil$, and the encoding introduces ℓ auxiliary variables $b_1, \dots, b_\ell$ [5]. With

$$L_{i,j} = \begin{cases} b_j, & c(i)_j = 1, \\ \neg b_j, & c(i)_j = 0, \end{cases} \tag{12}$$

the encoding consists of the clauses

$$\text{AMO}_{\text{bin}}(X) = \bigwedge_{i=1}^{m} \bigwedge_{j=1}^{\ell} (\neg x_i \vee L_{i,j}). \tag{13}$$

A true input forces the auxiliary variables to its code, and two simultaneously true inputs would require conflicting values for at least one auxiliary variable. The encoding uses ℓ auxiliary variables and $m\ell$ clauses; PySAT encodes the case $m = 2$ as a single negative clause.

Commander. The inputs are partitioned into groups G_t of size at most $b_N = \lceil N/\lfloor\sqrt{N}\rfloor\rceil$; each group receives a commander variable c_t, linked to its inputs by

$$(\neg c_t \vee \bigvee_{x\in G_t} x) \quad \text{and} \quad \bigwedge_{x\in G_t} (\neg x \vee c_t). \tag{14}$$

AMO is imposed pairwise on the commanders and inside each group (pairwise if $|G_t| \le 4$, binary otherwise). There is a single commander level [8]. For $N = 128$, a row has 11 groups of at most 12 inputs.

Product. The inputs are mapped injectively onto a $p \times q$ grid of auxiliary coordinate variables $U = \{u_1, \dots, u_p\}$ and $V = \{v_1, \dots, v_q\}$, with

$$p = \lceil\sqrt{m}\,\rceil, \qquad q = \lceil N/p\rceil. \tag{15}$$

Input x_k occupies grid position $(r(k), s(k))$ in row-major order, and implies its row and column variables:

$$\mathrm{AMO}_{\mathrm{prod}}(X) = \bigwedge_{k=1}^{m} \left[(\neg x_k \vee u_{r(k)}) \wedge (\neg x_k \vee v_{s(k)})\right] \wedge \mathrm{AMO}(U) \wedge \mathrm{AMO}(V). \tag{16}$$

The AMO constraints on U and V use the pairwise encoding when $m \le 10$ and the binary encoding otherwise. Two distinct true inputs differ in at least one coordinate and therefore violate one of these constraints. The implementation follows Chen [4] without recursive decomposition. Because $m \le N$, the grid always has at least m positions; the conventional construction uses $q = \lceil m/p\rceil$, whereas our implementation uses N in place of m, so short diagonals receive unused coordinates.

Sequential counter. This encoding introduces prefix variables $s_1, \dots, s_{m-1}$ that carry the implications needed to exclude a second true input [15]. A true input in $x_1, \dots, x_i$ forces s_i true, but the clauses do not require s_i to equal that prefix disjunction in every satisfying auxiliary assignment:

$$(\neg x_1 \vee s_1), \quad (\neg s_{m-1} \vee \neg x_m), \tag{17}$$

$$(\neg x_i \vee s_i), \quad (\neg s_{i-1} \vee s_i), \quad (\neg s_{i-1} \vee \neg x_i) \qquad 2 \le i \le m-1. \tag{18}$$

This uses $3m-4$ clauses and $m-1$ auxiliary variables. Over the board it gives $12N^2-22N+8$ clauses and $5N^2-6N-2$ variables for $N \ge 4$: the fewest clauses of the five, with roughly five times as many variables as the board has cells.

4.2. Expected effects of encoding structure

All five encodings are correct (Appendix D) and maintain the same generalised arc consistency on the original inputs of an isolated AMO constraint (Section 2.2): in each of them, a true input x_i propagates to $\neg x_j$ for every other input of that constraint, as the propagation chains listed below show.

These input exclusions are the same, but the propagation depth δ_E and the number of auxiliary variables through which each derivation passes differ. Table 2 summarises δ_E together with the measured formula size at $N = 128$.

Table 2
Structure of the five implemented encodings. δ_E is the propagation depth, the largest number of unit-propagation steps from x_i to $\neg x_j$. Size columns are measured at $N = 128$; "growth" is the ratio of clause counts at $N = 128$ and $N = 64$, where a cubic formula predicts 8 and a quadratic one 4.

Encoding	Aux. per AMO	δ_E	Clauses	Variables	Growth
Pairwise	0	1	3,462,784	16,384	8.07
Binary	$\lceil\log_2 m\rceil$	2	447,828	21,234	4.69
Commander	$\approx m/b_N$ + local bits	3	352,868	44,176	4.81
Product	$p+q$ + local bits	3 or 4	249,040	41,766	3.95
Sequential	$m-1$	m	193,800	81,150	4.06

The paths follow from the clauses. Pairwise reaches $\neg x_j$ in one step. Binary goes through a code bit: $x_i \to L_{i,k} \to \neg x_j$. Commander goes through the commanders: $x_i \to c_t \to \neg c_u \to \neg x_j$. Product goes through a coordinate and its

AMO, which is itself binary for long lines. Sequential propagates along the prefix chain: $x_i \to s_i \to s_{i+1} \to \cdots \to s_{j-1} \to \neg x_j$, up to N steps on a row of a board of size N.

This structure suggests several expectations. Since pairwise grows cubically and the other encodings roughly quadratically, doubling N should multiply preparation cost by about eight for pairwise and by four to five for the others. Longer propagation paths should increase the work per decision, so pairwise should require the least propagation and sequential the most, with the gap widening as N grows; the paths of binary, commander, and product are too close to order them. Since the encodings support the same input-level deductions within an isolated AMO, solvers that simplify the formula during search should be less sensitive to the choice of encoding than Glucose3. Finally, when search is inexpensive, the ranking of encodings should follow their preparation cost.

5. Experimental design

Each batch covers four SAT solvers with five encodings and four native models at five sizes, repeated five times: $5 \times (4 \times 5 + 4) \times 5 = 600$ attempts. Every attempt runs in a fresh process with one solver thread, in a fixed order, with no warm-up. CaDiCaL and the native models use seed 0; the other SAT solvers use their PySAT defaults. Seeds are not varied, so repetitions measure timing noise, not independent search trials; indeed the recorded search counters are identical across repetitions and across batches.

We record two timers. `wall_s` is the elapsed time of the whole worker process, including Python start-up, imports, model construction, solving, validation, and teardown; the 120-second deadline applies to it. `total_s` covers only formula generation, loading, solving, and model extraction (for native models, model construction and the solver call). For SAT runs we also record the phase times and, where the PySAT interface exposes them, the numbers of decisions, conflicts, and propagations; these counters are available for Glucose3 and CaDiCaL. To keep failures in every runtime figure, we use resource PAR-2, where w_i is the worker wall time of attempt i:

$$P(A) = \frac{1}{|A|} \sum_{i \in A} \begin{cases} w_i, & \text{if attempt } i \text{ returns a validated placement,} \\ 2T, & \text{otherwise,} \end{cases} \qquad T = 120 \text{ s.} \tag{19}$$

for a set of attempts A. All runtimes in this report are PAR-2 values: over the 25 attempts of a configuration for the aggregate ranking (Table 3), and over the five attempts at one size everywhere else. Where all five attempts at a size time out, the tables show TO (PAR-2 = 240 s). Since only Glucose3–sequential at $N = 128$ fails, PAR-2 equals the plain mean everywhere else. Search counters are identical across repetitions. Every returned placement was independently validated. Appendix A describes the platform, the measurement boundaries, and the integrity checks.

6. Results

6.1. Overview

Of 600 attempts per batch, 595 return a valid placement. The other five are all Glucose3–sequential at $N = 128$, which times out in every repetition. There are no memory failures, errors, or invalid answers, and the 6144-MiB batch reproduces every outcome (Appendix B). Table 3 compares all 24 configurations. The following subsections examine the contributions of preparation, search, and process overhead to their overall performance. Figure 1 compares the five encodings within each SAT backend across sizes. The shared axes expose Glucose3's non-monotone search cost and sequential timeout, alongside the narrower runtime spreads of the other backends.

Run-to-run timing variation is small (median coefficient of variation 2.0% at 2048 MiB and 3.5% at 6144 MiB), and each SAT backend's best encoding is the same across repetitions, with a single exception for CryptoMiniSat (Appendix C).

6.2. Formula construction

Table 4 reports the phase times of CryptoMiniSat at $N = 128$. Pairwise produces the largest formula, with 3.46 million clauses, and requires almost two seconds for generation and loading alone, whereas each of the other encodings requires less than half a second. Pairwise also has the highest memory usage.

The loading measurements agree with the clause counts. When N doubles from 64 to 128, pairwise loading time grows about eightfold, consistent with its cubic clause count. For the other encodings it grows by a factor of approximately four to five. The clause counts in Section 4 thus provide a useful estimate of loading growth. Absolute

Table 3
Fixed configurations across all five sizes. Each budget contributes 25 attempts per row. PAR-2 is in seconds, using worker wall time and a 240-second penalty for each failure; lower is better.

Backend / encoding	SAT/attempts	PAR-2, 2048 MiB	PAR-2, 6144 MiB
Glucose3 / Pairwise	25/25	1.3265	1.2888
Glucose3 / Binary	25/25	6.6487	6.5892
Glucose3 / Commander	25/25	1.0011	0.9712
Glucose3 / Product	25/25	1.3654	1.3282
Glucose3 / Sequential	20/25	51.5830	51.5819
CaDiCaL / Pairwise	25/25	2.2395	2.1520
CaDiCaL / Binary	25/25	1.3251	1.3124
CaDiCaL / Commander	25/25	1.4436	1.4246
CaDiCaL / Product	25/25	1.4158	1.3897
CaDiCaL / Sequential	25/25	1.5994	1.5590
Kissat / Pairwise	25/25	2.9653	2.9594
Kissat / Binary	25/25	1.6456	1.6258
Kissat / Commander	25/25	1.2140	1.1962
Kissat / Product	25/25	1.1382	1.1250
Kissat / Sequential	25/25	1.2254	1.2134
CryptoMiniSat / Pairwise	25/25	1.2563	1.2444
CryptoMiniSat / Binary	25/25	0.8888	0.8693
CryptoMiniSat / Commander	25/25	0.9262	0.9011
CryptoMiniSat / Product	25/25	0.8711	0.8552
CryptoMiniSat / Sequential	25/25	0.8841	0.8633
CP Optimizer	25/25	0.5002	0.4925
CPLEX MIP	25/25	0.1700	0.1718
Gurobi MIP	25/25	0.2898	0.2747
CP-SAT	25/25	0.9906	1.1334

Table 4
CryptoMiniSat at $N = 128$ and 2048 MiB: formula size, mean phase times (seconds), and maximum worker RSS (MiB). All five attempts succeeded, so these means equal PAR-2. Total also includes model extraction.

Encoding	Clauses	Variables	Gen.	Load	Solve	Total	RSS
Pairwise	3462784	16384	0.9773	0.9141	0.0421	1.9338	531.17
Binary	447828	21234	0.1841	0.1334	0.0693	0.3871	174.38
Commander	352868	44176	0.2643	0.1351	0.1396	0.5392	188.10
Product	249040	41766	0.1276	0.0894	0.0961	0.3134	167.75
Sequential	193800	81150	0.1389	0.0967	0.1383	0.3742	168.67

preparation costs also depend on the implementation: sequential has the fewest clauses, yet product is quicker to generate and load.

6.3. Search

Short paths reduce propagation work. Although pairwise is the most expensive encoding to construct, it has the shortest search phase for both Glucose3 and CryptoMiniSat. The counters in Table 5 help explain the Glucose3 result. At $N = 64$, pairwise requires about 8 propagations per decision, compared with almost 300 for sequential. This is consistent with the path lengths at the two extremes: the shortest paths coincide with the least propagation work, and the longest paths with the most.

Binary shows the limits of this account. Its paths are short, yet Glucose3 encounters substantially more conflicts with it than with pairwise. Each code bit is shared across a line, so a decision on that bit can affect many cells. This coupling offers a possible explanation for the additional search work, although the counters do not establish the mechanism.

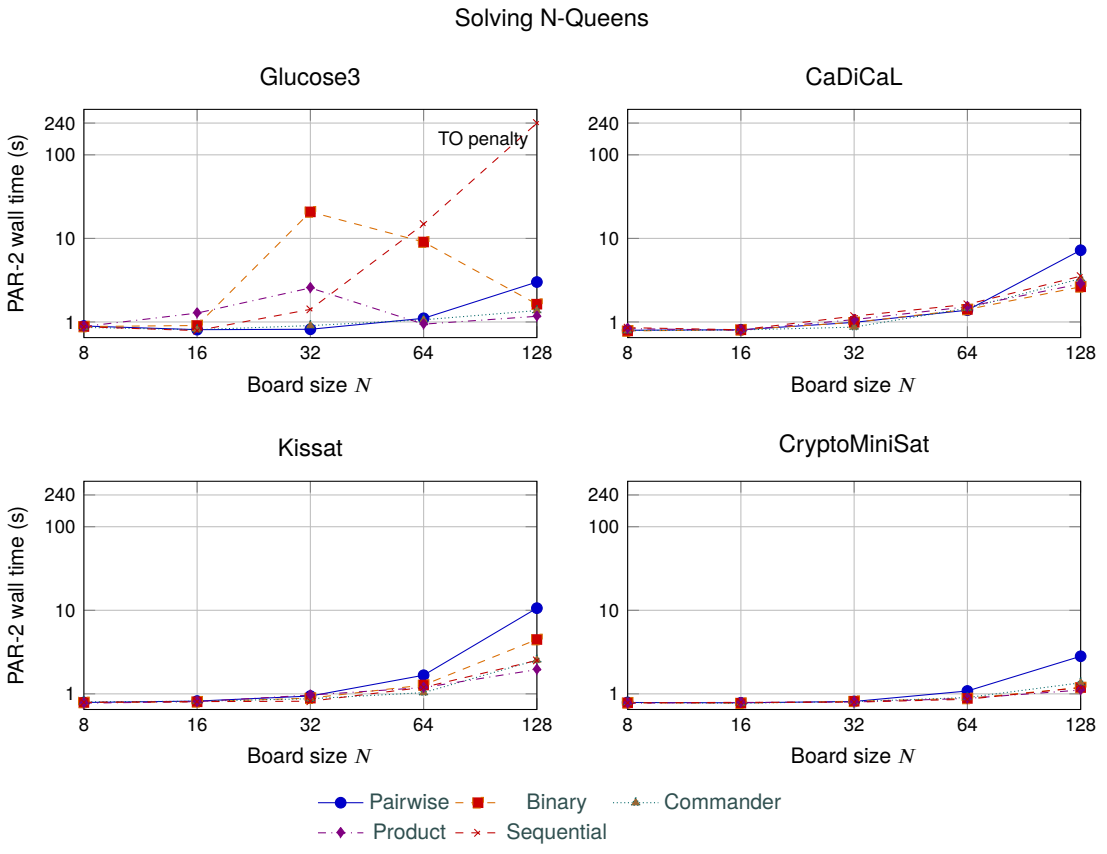


Figure 1: Encoding comparison within each SAT backend at 2048 MiB, with shared logarithmic axes. Each point is PAR-2 worker wall time over five attempts. Glucose3–sequential at $N = 128$ has five timeouts and is plotted at the 240-second penalty, not a completed runtime; all other points are means of five successful attempts. Lines connect measured sizes and do not imply monotone scaling.

Table 5
Search counters at 2048 MiB (identical in all repetitions and in the 6144-MiB batch). TO marks a timeout. Propagations per decision are for Glucose3 at $N = 64$.

	Glucose3 conflicts			Glucose3	CaDiCaL conflicts		
Encoding	$N = 32$	$N = 64$	$N = 128$	prop./dec.	$N = 32$	$N = 64$	$N = 128$
Pairwise	233	441	553	8.4	6,050	0	6,288
Binary	234,682	69,736	2,307	173.3	6,108	6,711	6,066
Commander	3,205	2,523	780	24.7	982	6,096	6,102
Product	25,810	1,285	729	42.5	6,055	6,659	6,234
Sequential	8,547	53,547	TO	293.8	7,803	7,054	6,080

Sequential's growing cost and the Glucose3 timeout. Glucose3's search cost with sequential rises with board size, and at $N = 128$ every attempt times out. The growing prefix chains provide a structural explanation for increasing propagation work, consistent with the observed trend. The other three solvers complete the same formula within the deadline, showing that this difficulty also depends on the solver.

Decreasing difficulty on larger boards. For binary, product, and commander, Glucose3 needs *fewer* conflicts on larger boards. Glucose3–binary, for example, requires 20.8 seconds at $N = 32$ but only 1.6 seconds at $N = 128$. One possible explanation is the rapid growth in the number of N-Queens solutions [14], which may make a satisfying placement easier to reach. Solution counts alone, however, do not determine the likelihood that a partial assignment

can be extended. The fixed-seed runs establish the timing and conflict trends; they do not isolate their cause. Sequential shows the opposite trend, consistent with its increasing propagation cost.

Clustering of CaDiCaL's conflict counts. CaDiCaL exhibits a different pattern. Many cases from $N = 32$ upward cluster near 6,000 conflicts, across encodings and board sizes, although there are exceptions (Table 5). This clustering suggests a possible connection to its internal schedule. Periodic rephasing, which can include a short local-search phase [3], is one candidate, given N-Queens' suitability for local search [9]. We did not record the data needed to confirm this. Binary has the best aggregate performance for CaDiCaL; the similar conflict counts suggest that preparation and work per conflict help distinguish the encodings.

Sensitivity of solvers to the encoding. The aggregate scores show that the other solvers are less sensitive to the encoding. For Glucose3, the gap between the best and worst encoding is approximately a factor of fifty; for CaDiCaL, Kissat, and CryptoMiniSat it is less than a factor of three (Table 3). These scores include preparation, process overhead, and timeout penalties, so they do not isolate the effect of simplification. Pairwise has the worst aggregate score for each of the latter three solvers, consistent with the cost of preparing and processing its much larger formula.

Dominance of preparation when search is inexpensive. CryptoMiniSat searches every encoding quickly, so its results illustrate the case in which preparation dominates. Its aggregate encoding ranking matches the preparation-time ranking: product, sequential, binary, commander, and pairwise. Product has the best overall score even though pairwise is faster during search; its lower preparation cost more than offsets that difference.

Dependence of the best encoding on the solver. No encoding has the best aggregate score for every solver. Commander leads for Glucose3, combining relatively low conflict counts with a smaller formula than pairwise. Binary leads for CaDiCaL, while product leads for Kissat and CryptoMiniSat. Product's low preparation cost helps explain its advantage, particularly when search is inexpensive. Each solver's best encoding is the same at both memory budgets and, with one CryptoMiniSat exception, in every repetition (Appendix C). The margins, however, differ in practical weight. For CryptoMiniSat, product wins 9 of the 10 repetitions, but only by about 0.01 seconds per attempt over sequential and binary (0.871, 0.884 and 0.889 seconds at 2048 MiB), whereas pairwise is 44% slower (1.256 seconds). Apart from pairwise, CryptoMiniSat is therefore practically insensitive to the encoding, whereas for Glucose3 the choice decides whether $N = 128$ is solved at all.

6.4. Process overhead and native models

Every attempt incurs start-up overhead before solving begins. On the smallest board, a SAT run requires approximately 0.8 seconds, CP Optimizer and CP-SAT approximately 0.5 seconds, and the two MIP solvers 0.1–0.2 seconds. For the SAT runs, this overhead accounts for most of the total time and is independent of the encoding.

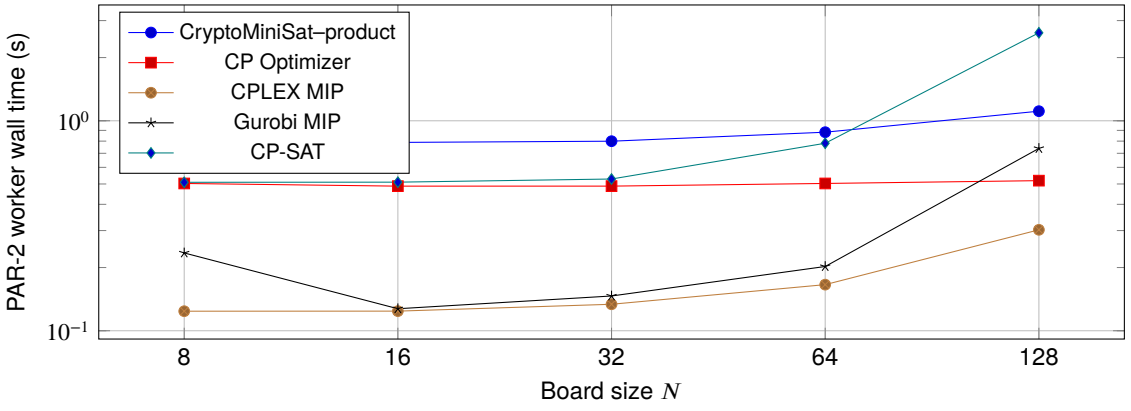


Figure 2: PAR-2 worker wall time at 2048 MiB for CryptoMiniSat–product and the four native models. Each point summarizes five attempts, all successful. The SAT configuration stays fixed across sizes; its selection is descriptive.

The native models require no explicit AMO encoding (Figure 2, Table 6). CP Optimizer describes the board with three constraints instead of hundreds of thousands of clauses, and solves $N = 128$ within a few hundredths of a second.

CPLEX MIP is the fastest configuration overall by worker wall time. Its assignment structure gives it an integral row–column relaxation before the diagonal constraints are added, a possible structural advantage. The measurements do not isolate this advantage from other solver features. Gurobi MIP is also fast inside the solver, but at $N = 128$ its complete run is considerably longer, for reasons outside the solver timer. The runtime of CP-SAT increases more steeply with N. Its internal translation of parts of the model into Boolean reasoning [11] may contribute, but we did not measure that cost separately.

The choice of timer affects this comparison. Measured by solver time alone, CP Optimizer is faster than CPLEX MIP; measured over the whole run, CPLEX MIP is faster owing to its lower start-up overhead.

Table 6
PAR-2 worker wall time at $N = 128$ (seconds) over five attempts, with caps of 2048 and 6144 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Backend / encoding	PAR-2, 2048 MiB	PAR-2, 6144 MiB
Glucose3 / Pairwise	3.0055	2.9897
Glucose3 / Binary	1.6326	1.6043
Glucose3 / Commander	1.3696	1.3503
Glucose3 / Product	1.1691	1.1935
Glucose3 / Sequential	TO	TO
CaDiCaL / Pairwise	7.2209	6.8501
CaDiCaL / Binary	2.6365	2.6864
CaDiCaL / Commander	3.2952	3.2658
CaDiCaL / Product	2.8601	2.8208
CaDiCaL / Sequential	3.5351	3.4846
Kissat / Pairwise	10.6051	10.6541
Kissat / Binary	4.4703	4.4507
Kissat / Commander	2.5426	2.5319
Kissat / Product	1.9523	1.9318
Kissat / Sequential	2.5133	2.4869
CryptoMiniSat / Pairwise	2.8175	2.8325
CryptoMiniSat / Binary	1.1902	1.1734
CryptoMiniSat / Commander	1.3559	1.3314
CryptoMiniSat / Product	1.1109	1.0915
CryptoMiniSat / Sequential	1.1901	1.1522
CP Optimizer	0.5186	0.5065
CPLEX MIP	0.3025	0.2808
Gurobi MIP	0.7384	0.7384
CP-SAT	2.6239	3.3363

7. Discussion

The benchmark supports a practical account of the costs of an AMO encoding. Within an isolated AMO constraint, all five propagate a true original input to the exclusion of the other original inputs, yet the clauses and intermediate steps needed to reach that conclusion differ. Clause-count growth closely tracks loading-time growth, while propagation paths help explain the large difference in Glucose3's work per decision between pairwise and sequential. Together, these observations connect formula structure to measurable costs without making path length a complete model of search.

The balance between preparation and search depends on the solver. Glucose3's results show the cost of long propagation chains, but also the importance of conflict frequency: binary has short paths and still requires substantial search. CaDiCaL, Kissat, and CryptoMiniSat have narrower aggregate performance spreads across encodings. For CryptoMiniSat, inexpensive search makes preparation the principal distinction, and product's low preparation cost gives it the best SAT score. A compact encoding is therefore most valuable when the solver can search it efficiently.

The search results also invite explanations based on N-Queens itself. The growing abundance of solutions may help account for the easier larger instances, while suitability for local search makes rephasing a plausible explanation for CaDiCaL's clustered conflict counts. Both remain hypotheses. They suggest useful follow-up measurements and

reinforce the scope of the comparison: a benchmark measures an encoding *on a problem*. The same encodings could rank differently on completion instances or other problems with more restricted solution sets.

The native models put these trade-offs in a broader practical context. At the tested sizes, process overhead often exceeds SAT search time. Native interfaces allow the model to be stated compactly, and the leading native implementations achieve the lowest worker wall times. Practitioners selecting an approach for this workload should therefore consider the complete process from model construction to a validated answer. Within the SAT approach, the results identify which encodings work best with each solver under the measured conditions.

Limitations. Repetitions use fixed seeds and order, so they describe timing noise rather than independent trials; the largest board in these two batches is $N = 128$; their boards are unrestricted and of even, power-of-two size. Search counters are available only for Glucose3 and CaDiCaL, so the explanations for Kissat and CryptoMiniSat rest on phase times. The rephasing account of CaDiCaL's conflict count is a hypothesis consistent with the data, not a measured fact. Worker wall time includes interface-specific start-up costs, so the cross-paradigm comparison evaluates complete implementations rather than solver algorithms.

8. Conclusion

The five AMO encodings preserve the same board solutions and propagate the same original-input exclusions within an isolated AMO constraint, yet their measured costs differ by more than an order of magnitude. The comparison reveals two recurring costs: preparing the formula and searching it. Pairwise's cubic clause growth makes preparation expensive at larger sizes; sequential requires substantially more propagation work with Glucose3 and is the only encoding to time out, at $N = 128$. The solver determines how these costs balance. Commander has the best aggregate score for Glucose3, binary for CaDiCaL, and product for Kissat and CryptoMiniSat. CryptoMiniSat–product is the best SAT configuration overall (PAR-2 0.871 seconds), although its lead over CryptoMiniSat's sequential and binary encodings is only about 0.01 seconds per attempt; apart from pairwise, CryptoMiniSat is practically insensitive to the encoding. The fastest native implementation, CPLEX MIP, leads overall at 0.170 seconds. For these unrestricted boards, the results indicate that the representation and the solver should be chosen together, taking preparation and process overhead into account. Extending this repeated protocol to larger and odd board sizes, completion instances, varied seeds, and direct measurements of solver scheduling would test how far this account extends.

Data availability

The benchmark code, including the encoder implementations, the solver interfaces, and the analysis script that computes the reported statistics, is available at `https://github.com/Oxigo-1212/SAT-benchmark`; the repository documentation describes how to rerun the benchmark. The raw per-attempt results of both batches, from which every table and figure in this report is derived, are available from the author upon reasonable request.

A. Experimental setup and provenance

A.1. Platform and software

Both batches were run on an AMD Ryzen 7 6800HS processor (16 logical CPUs, 14.3 GiB memory) under Linux 7.2.9 with CPython 3.14.0. The SAT solvers were accessed through python-sat 1.9.dev15 (classes `Glucose3`, `Cadical300`, `Kissat404`, and `CryptoMinisat`) and pycryptosat 5.11.21; the native models used cplex 22.2.0.1 with docplex 2.32.264, gurobipy 13.0.3, and ortools 9.15.6755. Pairwise, binary, and sequential were generated with PySAT `CardEnc`; commander and product were implemented as described in Section 4. The two batches were run consecutively on 19 October 2026.

A.2. Measurement boundaries

Each attempt runs in a separate worker process. Worker wall time includes interpreter start-up, imports, model construction, solving, validation of the returned placement, and teardown. Wall times recorded for timed-out attempts may slightly exceed 120 seconds because process termination takes time; they are not solve times and enter PAR-2 only as the 240-second penalty. The model-and-solver timer excludes imports, validation, and teardown, and for Gurobi also excludes licence initialisation. Peak memory is the resident set size of the worker process and excludes external solver processes such as that of CP Optimizer. The memory cap is a per-process virtual address-space limit (`RLIMIT_AS`).

A.3. Integrity of the results

Several safeguards support the reliability of the reported values. Each batch archives an exact copy of the benchmark source together with SHA-256 checksums of every file, so the measured code can be identified independently of later changes to the repository. All analysis is computed from the raw per-attempt records rather than from intermediate summaries, and the analysis code is taken from the same archived source. Before any value is computed, the analysis verifies the archived checksums, confirms that each batch contains exactly the 600 expected attempts, checks that the two raw record formats agree on every outcome, and independently validates all 1,190 returned placements for row, column, and diagonal uniqueness. Regenerating the analysis twice produces byte-identical outputs. Finally, for Glucose3 and CaDiCaL, the solvers that report search counters, these counters are identical across all repetitions and both batches, which confirms that their search was deterministic under the fixed seeds.

B. Memory-budget comparison

All 600 matched attempts have the same outcome at both budgets, all SAT pairs have identical clause and variable counts, and all recorded search counters (245 pairs) match exactly. Maximum worker RSS is 1046.40 MiB at 2048 MiB and 1046.65 MiB at 6144 MiB, in both cases from CaDiCaL–pairwise at $N = 128$. The mean wall-time difference (6144 minus 2048 MiB) is -0.0162 seconds and the median -0.0005 seconds. The largest configuration-specific change is CP-SAT at $N = 128$, whose PAR-2 rises from 2.62 to 3.34 seconds; because the batches ran in a fixed sequence, this cannot be attributed to the memory cap. The rankings in Table 3 are the same at both budgets for every solver's best encoding.

C. Timing variation

For each configuration and size with five successful attempts we compute the coefficient of variation of the worker wall times $w_1, \ldots, w_5$,

$$\mathrm{CV} = \frac{s}{\bar{w}}, \qquad \bar{w} = \frac{1}{5}\sum_{i=1}^{5} w_i, \qquad s = \sqrt{\frac{1}{4}\sum_{i=1}^{5}(w_i - \bar{w})^2}, \tag{20}$$

using the `wall_s` field of the successful records in `data/proposed-2048-results.json` and `data/proposed-6144-res` (119 combinations per batch; Glucose3–sequential at $N = 128$ is excluded because it never succeeds). The median CV is 2.0% at 2048 MiB and 3.5% at 6144 MiB, and 108 and 109 of the 119 combinations respectively stay below 10%. The largest values occur in sub-second runs at $N = 8$, where process start-up dominates (Gurobi at $N = 8$ ranges from 0.12 to 0.70 seconds at 2048 MiB), and in CP-SAT at $N = 128$. Computing the aggregate score separately for each repetition, the best encoding of every SAT backend is the same in all five repetitions of both batches, except that sequential overtakes product for CryptoMiniSat in one repetition at 6144 MiB. The per-backend winners are therefore stable across repetitions.

D. Correctness and validation

D.1. Minimal and extended formulations

The minimal formulation replaces column exactly-one with at-most-one:

$$F_{\min} = \bigwedge_{r} \mathrm{EO}(R_r) \wedge \bigwedge_{c} \mathrm{AMO}(C_c) \wedge \bigwedge_{D \in \mathcal{D}} \mathrm{AMO}(D), \qquad F_{\mathrm{ext}} = F_{\min} \wedge \bigwedge_{c} \mathrm{ALO}(C_c). \tag{21}$$

Row exactly-one places N queens; column at-most-one places them in N distinct columns, so every column is occupied. The added column clauses preserve projected solutions and add no variables. Only the extended formulation is timed here.

D.2. Projected equivalence argument

For each AMO construction, correctness requires two directions: its CNF must forbid two true inputs, and every input assignment with zero or one true literal must admit an auxiliary extension. Pairwise clauses enforce the first direction directly. Distinct binary patterns make two true inputs inconsistent; with one true input, choose its pattern, and with none, the selector bits are unrestricted. For the sequential counter, set each prefix variable to the disjunction

of its input prefix. This satisfies every clause when at most one input is true; the exclusion clauses prevent a second true input after an occupied prefix.

Commander groups partition the inputs. Each commander is equivalent to the disjunction of its group, local AMO forbids two true inputs within a group, and commander AMO forbids two occupied groups. A valid input assignment extends by selecting only its occupied group's commander. For product, $pq \geq N \geq m$ ensures distinct coordinate pairs for all inputs. Two distinct true inputs must differ in a row or column and violate a coordinate AMO. With one true input, select only its two coordinates; with none, set all coordinates false. Unused coordinates do not exclude valid inputs. Local binary or pairwise constraints admit the extensions just described. Since auxiliary identifiers are disjoint between constraints, these extensions combine across all board lines. This establishes projected equivalence for the specified constructions; the tests check their implementation on a bounded domain.

D.3. Saved witnesses and bounded semantic evidence

For each saved successful attempt, the regeneration check verifies exactly N column indices in $\{1, \ldots, N\}$, unique columns, and unique values of $r + q_r$ and $r - q_r$; all 1,190 saved placements pass. The file `data/semantic-validation.json` documents 184,479 exhaustive AMO assumption checks using Glucose3 (input lengths 0–11, positive and negative literals, custom board-size parameters) and exact agreement of 80 projected solution sets (five encodings, two formulations, sizes 1–8) with an independent backtracking oracle, with solution counts $1, 0, 0, 2, 10, 4, 40, 92$. The tested encoder hashes match those archived with the selected batches. Earlier native integration checks passed for CP Optimizer, CPLEX MIP, and CP-SAT on sizes 1–4 and 8; Gurobi was not tested then because of a license-host error, but all 50 of its attempts in the present batches return validated placements.

E. Complete per-size runtime tables

The following tables give PAR-2 worker wall time over five attempts for all 120 size/configuration combinations per budget; TO denotes five timeouts. The per-size CSV additionally provides medians, IQRs, minima, maxima, phase timings, sample counts, and maximum worker RSS.

E.1. 2048-MiB budget

Table 7
Glucose3 PAR-2 worker wall time (seconds) over five attempts at 2048 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Encoding	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
Pairwise	0.8947	0.8089	0.8176	1.1060	3.0055
Binary	0.8819	0.9071	20.7666	9.0551	1.6326
Commander	0.8609	0.8081	0.9051	1.0616	1.3696
Product	0.8827	1.2773	2.5563	0.9416	1.1691
Sequential	0.8765	0.7888	1.4145	14.8352	TO

Table 8
CaDiCaL PAR-2 worker wall time (seconds) over five attempts at 2048 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Encoding	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
Pairwise	0.7963	0.8066	0.9893	1.3842	7.2209
Binary	0.7816	0.8065	0.9895	1.4117	2.6365
Commander	0.7889	0.8034	0.8672	1.4632	3.2952
Product	0.8320	0.7976	1.0683	1.5212	2.8601
Sequential	0.8546	0.8067	1.1755	1.6252	3.5351

Table 9
Kissat PAR-2 worker wall time (seconds) over five attempts at 2048 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Encoding	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
Pairwise	0.7888	0.8195	0.9420	1.6710	10.6051
Binary	0.7890	0.8037	0.8890	1.2758	4.4703
Commander	0.8025	0.8161	0.8745	1.0343	2.5426
Product	0.7743	0.8067	0.9675	1.1901	1.9523
Sequential	0.7920	0.8080	0.8161	1.1975	2.5133

Table 10
CryptoMiniSat PAR-2 worker wall time (seconds) over five attempts at 2048 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Encoding	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
Pairwise	0.7889	0.7814	0.8113	1.0826	2.8175
Binary	0.7816	0.7741	0.8162	0.8818	1.1902
Commander	0.7889	0.7741	0.8081	0.9038	1.3559
Product	0.7742	0.7889	0.7994	0.8822	1.1109
Sequential	0.7816	0.7917	0.7963	0.8609	1.1901

Table 11
Native-model PAR-2 worker wall time (seconds) over five attempts at 2048 MiB. All attempts succeeded.

Backend	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
CP Optimizer	0.5028	0.4883	0.4884	0.5029	0.5186
CPLEX MIP	0.1239	0.1240	0.1337	0.1658	0.3025
Gurobi MIP	0.2347	0.1273	0.1464	0.2023	0.7384
CP-SAT	0.5093	0.5101	0.5280	0.7816	2.6239

E.2. 6144-MiB budget

Table 12
Glucose3 PAR-2 worker wall time (seconds) over five attempts at 6144 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Encoding	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
Pairwise	0.7752	0.7727	0.8090	1.0973	2.9897
Binary	0.7615	0.8818	20.6820	9.0162	1.6043
Commander	0.7597	0.7902	0.8924	1.0632	1.3503
Product	0.7744	1.1965	2.5402	0.9366	1.1935
Sequential	0.9095	0.7743	1.4040	14.8217	TO

Table 13
CaDiCaL PAR-2 worker wall time (seconds) over five attempts at 6144 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Encoding	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
Pairwise	0.7805	0.7921	0.9650	1.3722	6.8501
Binary	0.7605	0.7889	0.9621	1.3642	2.6864
Commander	0.7679	0.7814	0.8574	1.4506	3.2658
Product	0.7584	0.7839	1.0761	1.5096	2.8208
Sequential	0.7727	0.8026	1.1442	1.5908	3.4846

Table 14
Kissat PAR-2 worker wall time (seconds) over five attempts at 6144 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Encoding	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
Pairwise	0.7646	0.8071	0.9397	1.6316	10.6541
Binary	0.7720	0.8016	0.8546	1.2502	4.4507
Commander	0.7583	0.7993	0.8620	1.0296	2.5319
Product	0.7761	0.7970	0.9434	1.1765	1.9318
Sequential	0.7729	0.8031	0.8143	1.1899	2.4869

Table 15
CryptoMiniSat PAR-2 worker wall time (seconds) over five attempts at 6144 MiB. TO: all five attempts timed out, giving PAR-2 = 240 s.

Encoding	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
Pairwise	0.7742	0.7647	0.7953	1.0553	2.8325
Binary	0.7735	0.7492	0.7888	0.8617	1.1734
Commander	0.7542	0.7606	0.7826	0.8765	1.3314
Product	0.7679	0.7726	0.7961	0.8481	1.0915
Sequential	0.7616	0.7607	0.7897	0.8521	1.1522

Table 16
Native-model PAR-2 worker wall time (seconds) over five attempts at 6144 MiB. All attempts succeeded.

Backend	$N = 8$	$N = 16$	$N = 32$	$N = 64$	$N = 128$
CP Optimizer	0.4810	0.4927	0.4946	0.4874	0.5065
CPLEX MIP	0.1239	0.1336	0.1401	0.1805	0.2808
Gurobi MIP	0.1207	0.1393	0.1666	0.2087	0.7384
CP-SAT	0.4882	0.5084	0.5420	0.7922	3.3363

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI), pp. 399–404.

[2] Audemard, G., Simon, L., 2018. On the Glucose SAT solver. International Journal on Artificial Intelligence Tools 27, 1840001. doi:10.1142/S0218213018400018.

[3] Biere, A., Fazekas, K., Fleury, M., Heisinger, M., 2020. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020, in: Proceedings of SAT Competition 2020: Solver and Benchmark Descriptions, pp. 51–53.

[4] Chen, J., 2010. A new SAT encoding of the at-most-one constraint, in: Proceedings of the Ninth International Workshop on Constraint Modelling and Reformulation (ModRef).

[5] Frisch, A.M., Giannaros, P.A., 2010. SAT encodings of the at-most-k constraint: Some old, some new, some fast, some slow, in: Proceedings of the Ninth International Workshop on Constraint Modelling and Reformulation (ModRef).

[6] Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n-queens completion. Journal of Artificial Intelligence Research 59, 815–848. doi:10.1613/jair.5512.

[7] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437. doi:10.1007/978-3-319-94144-8_26.

[8] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects, in: Proceedings of the International Workshop on Constraints in Formal Verification. URL: https://www.cs.cmu.edu/~wklieber/papers/2007_efficient-cnf-encoding-for-selecting-1.pdf.

[9] Minton, S., Johnston, M.D., Philips, A.B., Laird, P., 1992. Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems. Artificial Intelligence 58, 161–205. doi:10.1016/0004-3702(92)90007-K.

[10] Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S., 2001. Chaff: Engineering an efficient SAT solver, in: Proceedings of the 38th Design Automation Conference, pp. 530–535. doi:10.1145/378239.379017.

[11] Ohrimenko, O., Stuckey, P.J., Codish, M., 2009. Propagation via lazy clause generation. Constraints 14, 357–391. doi:10.1007/s10601-008-9064-x.

[12] Philipp, T., Steinke, P., 2015. PBLib – a library for encoding pseudo-boolean constraints into CNF, in: Theory and Applications of Satisfiability Testing – SAT 2015, Springer. pp. 9–16. doi:10.1007/978-3-319-24318-4_2.

[13] Rivin, I., Vardi, I., Zimmermann, P., 1994. The n-queens problem. The American Mathematical Monthly 101, 629–639. doi:10.1080/00029890.1994.11997004.

[14] Simkin, M., 2023. The number of n-queens configurations. Advances in Mathematics 427, 109127. doi:10.1016/j.aim.2023.109127.

[15] Sinz, C., 2005. Towards an optimal CNF encoding of Boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. doi:10.1007/11564751_73.

[16] Soos, M., Nohl, K., Castelluccia, C., 2009. Extending SAT solvers to cryptographic problems, in: Theory and Applications of Satisfiability Testing – SAT 2009, Springer. pp. 244–257. doi:10.1007/978-3-642-02777-2_24.

[17] Sosič, R., Gu, J., 1994. Efficient local search with conflict minimization: A case study of the n-queens problem. IEEE Transactions on Knowledge and Data Engineering 6, 661–668. doi:10.1109/69.317698.